

# Use Of Supervised and Semi-Supervised Learning Models For Synthetic Data Generation Used In Range Estimation

1<sup>st</sup> Michail Pavlidis  
Dept. of Electronic Systems  
Aalborg University  
Aalborg, Denmark  
mpavli24@student.aau.dk

2<sup>nd</sup> Jonas Pedesk  
Dept. of Electronic Systems  
Aalborg University  
Aalborg, Denmark  
jpedes20@student.aau.dk

**Abstract**—This article investigates the possibility of using Supervised and Semi-Supervised machine learning models, to learn the distribution of real datasets, with the purpose of creating synthetic data of similar distributions hinged upon a distance-label that is used the condition for the models to work toward. This is done through two approaches, one using autoencoders, specifically a standard Autoencoder (AE), a Variational Autoencoder (VAE) and a Conditional Variational Autoencoder (CVAE). The second approach using a conditional generative adversarial network approach, trained on two different loss functions to compare how the loss and generated data might differ between the two. The performance of the models are measured in their loss, as well as the KL-divergence of the real data against the synthetic data. Here a lower divergence is preferred. This work has shown that these approaches are more than capable of creating data that is in-line with the distributional properties of the real data.

**Index Terms**—Supervised Learning, Semi-Supervised Learning, Bluetooth, Channel Sounding, Autoencoders, Generative, GAN, KL-Divergence.

## I. INTRODUCTION

Bluetooth Channel Sounding provides measurements that can be used in order to estimate the distance between two connected devices [1]. Measurements can be collected by using Phase-Based Ranging (PBR), Round Trip Time (RTT) or both, in order to estimate the distance between devices [1]. Previous experiments with Machine Learning-based algorithms have demonstrated highly accurate results by precise distance measurement. Such examples can be seen in [2], where machine learning models such as linear regression, decision trees, random forests and neural networks were used in order to estimate the distance between devices and improving proximity classification<sup>1</sup>.

In [3] a Neural Network optimizes range estimation by learning from raw channel sounding data, enabling adaptive feature extraction and robustness to environmental variations outperforming traditional methods that have been used for range estimation [3].

<sup>1</sup>Proximity classification is a term where the Bluetooth device will try to identify nearby objects or obstacles and provides valuable contextual knowledge to the user [2]

However, due to the multi path effect most of the data can become unreliable or too noisy, because of phase distortions or amplitude variations that can degrade the data accuracy [4]. Moreover, obtaining a sufficiently large and diverse labeled dataset poses a significant challenge, making it difficult to properly train the model, while the labeling process can be time consuming.

In this paper, it is analyzed whether it is possible to create synthesized data based on different supervised and semi-supervised models that will have a similar data distribution as the original. The reason behind this, is that there could be cases where the measurements can be limited and not enough data have been acquired.

## II. BACKGROUND

Channel Sounding is a feature in Bluetooth communications ensuring reliable communication between devices [1]. In wireless communications both devices communicate via wireless signals which can propagate via multiple paths [4]. In order to estimate the distance between the devices, channel sounding uses two methods in order to provide enough data for an estimation [1].

- PBR (Phase-Based Ranging) Calculates the frequency difference between devices [5]. Distance  $d$  can be found by knowing the phase difference  $\Delta\phi$  between frequencies  $f_1$  and  $f_2$  and given that  $c$  is the speed of light, distance can be known in this equation [4]:

$$d = \frac{c}{4\pi} \cdot \frac{\Delta\phi}{f_2 - f_1} \quad (1)$$

- RTT (Round-Trip Time) that calculates the distance based on the time it takes for a packet to be send from one device to another and return back to the first device. RTT is described as follows:

$$RTT = 2ToF = (ToA_I - ToD_I) - (ToD_R - ToA_R) \quad (2)$$

Where  $ToF$  is known as Time of Flight,  $ToA_I$  and  $ToA_R$  are the Times of Arrival, while  $ToD_I$  and  $ToD_R$  are the times

of departure from both the initiator and reflector devices [5]. Based from equation (2) distance can be calculated as:

$$d = \frac{c \cdot RTT}{2} \quad (3)$$

Both RTT and PBR make it easier to collect bluetooth data, but the resulting dataset is typically very large, making manual labeling prohibitively time-consuming, while these methods do not account for noisy data.

### III. METHODOLOGY

Before training the models it is necessary to determine the key parameters that are of importance. Several datasets were provided by **SDRC** (Samsung Denmark Research Center) which included data from different environments by using two phone devices.

#### A. Environment and Features

For each environment, distance was measured through different ranges. These measurements included the movement of one of the devices, which can be seen in the *distance\_cm* feature. Most environments had different variables such as range and capture environments differ. For that reason this paper focuses only on office\_rot datasets while also ignoring the moving labels. While the distance traveled in each environment is given by SDRC, it is not included in the datasets. Since any estimation of the individual steps of the device would be uncertain, the "moving"-captures are ignored.

Out of all the parameters in the datasets key ones were handpicked based on, their lack of pre-processing and filtering, allowing to work on raw data. The key parameters can be seen in table I.

TABLE I: Parameters extracted from the JSON dataset

| Parameter                    | Description                                                                                                                                  |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Distance_cm</b>           | The distance between two phones in cm, which is manually typed into the phone. The value is also set to <i>moving</i> which will be ignored. |
| <b>initiator_packet_rssi</b> | The RSSI signal from the initiator device.                                                                                                   |
| <b>reflector_packet_rssi</b> | The RSSI signal from the reflector device.                                                                                                   |
| <b>initiator_iq_data</b>     | Raw data from the initiator device.                                                                                                          |
| <b>reflector_iq_data</b>     | Raw data from the reflector device.                                                                                                          |
| <b>initiator_frequency</b>   | The frequency which the initiator sends signal                                                                                               |
| <b>reflector_frequency</b>   | The frequency which the receiver gets a signal                                                                                               |

As the features have been chosen and are ready to be processed this cannot be said for *iq\_data*. IQ represents the in-phase (I) and quadrature (Q) components of the received Bluetooth tones and are stored as strings of complex-valued numbers (e.g  $-437+87i$ ) [6]. To enable numerical processing and integration into a supervised model, each IQ entry must be parsed into its real and imaginary components and converted into a numerical representation.

#### B. Augmentation

After feature selection, data augmentation will be applied. Data augmentation is a method of altering the already existing data, making them similar to the original in terms of distribution by adding noise. This method increases the amount of data

, while exposing the model to different patterns and improve model optimization and its accuracy [7]. A gaussian approach was chosen since the uncertainty of noises make it difficult to choose any other methods of augmentation. Specifically this augmentation is applied to the RSSI, IQ and Frequency features, since they are unlabeled, while the distance label is kept away from augmentation. This ensures that augmentation is confined to the raw unclassified data.

#### C. Normalization

As the data distributions has been altered through augmentation, a normalization process applies across all features where the mean and standard deviation is calculated. The new normalized data  $X'$  are given as followed:

$$X' = \frac{X - \mu}{\sigma} \quad (4)$$

Where  $X$  is the raw data,  $\mu$  and  $\sigma$  the mean and the standard deviation of the raw data.

### IV. EXPERIMENTS

Several generative methodologies will be used, focused on autoencoders and GAN. In each method the models' parameters will be adjusted differently compared with the others. Below it is a list of the methods that will be used:

- Autoencoder (AE)
- Variational Autoencoder (VAE)
- Conditional Variational Autoencoder (CVAE)
- Conditional Generative Adversarial Network (CGAN)

#### A. Methodologies

Generative models attempt to reconstruct an approximation of the input [8]. This can be done with different methods, such as utilizing an encoder and decoder or an adversarial network. This paper will focus on autoencoders and GAN methods, since they can compress the data into lower-dimensional latent spaces [8]. This compression allows the model to learn meaningful representations directly from the raw high-dimensional data, enabling effective denoising, dimensionality reduction, and latent-space manipulation [8]. Three different approaches of autoencoders are examined which the supervised methods. These methods are:

1) *Autoencoder (AE)*: An AE is used to learn embeddings of unlabeled data for a given network configuration [8]. An autoencoder is consisted of two parts, an encoder  $z = e_{\phi}(x)$  that compresses the input data  $x$  from a higher dimensional space to a lower dimensional latent space  $z$ , and the decoder  $x' = d_{\theta}$  which converts the latent space back to higher dimensional space giving an approximation  $x'$  of the original input [8]. The decoder ensures that the latent space will capture most of the information from the dataset by forcing it to output an approximation of what was fed as input to the encoder.

AE aims to minimize the reconstruction loss during training, since the input is compressed and then it is reconstruct with little loss as possible [9]. Since data consists of continuous values choosing the mean squared error (MSE) reconstruction

seems ideal compared to binary cross entropy, which can be seen in equation (5).

$$MSE = \frac{1}{N} \sum_{i=1}^N (X - X')^2 \quad (5)$$

where  $X$  is the original input and  $X'$  the reconstructed input after passing through the encoder-decoder procedure [10].

2) *Variational Autoencoder (VAE)*: A VAE has the same concept of an AE, with the difference that the encoder outputs parameters of a pre-defined distribution in the latent space for every input [9]. It is a generative model that creates data similar in distribution to the input they are trained on and learn a continuous probabilistic representation of the underlying features [11].

Given the input sample  $x$ , the encoder does not output a single latent vector but instead it estimates the parameters of a gaussian distribution in a probabilistic manner, so the encoder will output a vector of means  $\mu$  and a vector of variances  $\sigma^2(x)$  defining the approximate posterior distribution  $q(z|x)$  as:

$$q(z|x) = N(\mu, \sigma^2) \quad (6)$$

The problem however is that  $z$  is referring to a single latent vector, but in the VAE the model samples an approximation of a latent distribution defined by the encoder [9]. For that reason the model uses a method known as the reparameterization trick which allows gradients to propagate through stochastic sampling, meaning that the latent variable can be expressed as:

$$z = \mu + \sigma \epsilon \quad (7)$$

with  $\epsilon$  being an auxillary noise variable  $\epsilon \sim N(0, 1)$ . Training the model involves minimizing the evidence lower bound (ELBO), which balances reconstruction accuracy and a regularization term that penalizes deviation of the latent distribution from a prior, typically a standard normal distribution [11]. The ELBO loss is consisted of a reconstruction term and a Kullback-Leibler Divergence ( $KLD$ ) of the encoding model to the prior distribution [12]. The ELBO equation is best described below:

$$L_{VAE} = -KLD(q(z|x)||p(z)) + E_{(z|x)}[\log p(x|z)] \quad (8)$$

where  $p(z)$  is the prior distribution,  $E_{(z|x)}$  is an approximation of the posterior which can be considered as the encoder, and  $p(x|z)$  the decoder [13].

3) *Conditional Variational Autoencoder (CVAE)*:: CVAE is an extension of VAE by using a condition; a labeled feature which helps in creating an output based on the condition [11]. While the conditional input can be added at different points of a model, in this paper it is used in both the encoder and decoder. The model is based on information  $Y$ , which will affect the model's output. The purpose of CVAE is to maximize  $\log p(x|Y)$ , for each observed variable  $x$ . This will transform equation (8) into:

$$L_{CVAE} = -KL(q(z|x, Y)||p(z|X, Y)) + E[\log p(x|z, Y)] \quad (9)$$

4) *Conditional Generative Adversarial Network*: Generative Adversarial Networks (GANs) are a type of machine learning method that combines two models, a generator ( $G$ ) and a discriminator ( $D$ ) [14]. Here  $G$  uses noise to generate a data vector and then send this to  $D$ , the ultimate goal being to fool  $D$  into classifying the data as real.  $D$  compares the input from  $G$  with the real-world input given to the network, and has the ultimate goal of catching all the fake data from  $G$ . This leads to a classification error that is fed to both  $G$  and  $D$  for their further tuning. The expected result being data generation that is close to indistinguishable from real data. A Conditional GAN (CGAN) operates on the same base principle, but with the added notion of being dependent on some condition. This condition can be seen as a sort of label. In this framework the generator is also fed the data. For this work the condition is the *distance\_cm* column. The model is trained in such a manner that the distribution of the data it generates, needs to follow the one in the real data dependent on the distance. Since the data spans 98 different distances, the distributions of the different features are expected to vary as well. This variation needs to be present in the synthetic data as well. For the CGAN, two different loss functions are used in an attempt to achieve favorable data generation. These being Binary Cross-Entropy and Wasserstein Loss with Gradient Penalty. For the BCE loss, the torch module `BCEWithLogitsLoss`, which is a loss function that combines BCE Loss and a Sigmoid layer. The loss is expressed using the equation found in equation (10) [15].

$$L(x, y) = L = \{l_1, \dots, l_N\}^T \quad (10)$$

$$l_n = -w_n[y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n))] \quad (11)$$

Where  $w_n$  is the weight for the corresponding entry,  $y_n$  is the generated value,  $x_n$  is the real value, and  $\sigma$  is the sigmoid function [15]. For the Wasserstein Loss, what is in theory happening is that a Conditional Wasserstein GAN (WGAN) is being created, with added gradient penalty. Wasserstein Loss is an expansion on the original Min-Max loss proposed by Goodfellow et al. in their 2014 paper. It deals with the risk of experiencing model collapse due to the model getting stuck early on in training. The loss functions for the discriminator and the generator can be found in equations (12) and (13) respectively [16].

$$L_D = -E_{x \sim p_d}[D(x)] + E_{\hat{x} \sim p_g}[D(\hat{x})] \quad (12)$$

$$L_G = -E_{\hat{x} \sim p_g}[D(\hat{x})] \quad (13)$$

Where  $E_{x \sim p_d}$  is the expectation that value  $x$  comes from the real data, and  $E_{\hat{x} \sim p_g}$  is the expectation that value  $\hat{x}$  is created by the generator. As mentioned the WGAN also introduces Gradient Penalty as away to optimize the generated values. This is done with the equation found in (14)

$$L_{GP} = \lambda \cdot E[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2] \quad (14)$$

What happens in equation (14) is that the 2-norm for the  $\hat{x}$  gradients for the discriminator is found. The reason 1 is subtracted is to penalize any deviation from the unit norm. The expression is squared to create symmetric penalty, and the mean is taken so that expectation is found for the entire batch. Lastly the result is multiplied by the strength hyperparameter, this controls the amount of impact the penalty has, which in this case is 10. This term is added to the discriminator loss.

### B. Comparing distribution

In order to compare the distribution between the original and the synthetic data, a Kullback-Leibler (KL) method will be used. While KL was mentioned in the VAE and CVAE, KL is also an information-theoretic measure that quantifies the difference between two probability distributions [?]. In this work, the distributions are estimated empirically for each feature using histogram-based density estimation with shared binning for both real and synthetic samples. The KL divergence is computed between the corresponding feature distributions, and a symmetric variant of the metric is reported to ensure numerical stability and a balanced comparison. Low KL divergence values indicate that model creates data that have similar distribution as the original, while a higher values proves the opposite. KL divergence is calculated as:

$$KL(P \parallel Q) = \sum_{x \in X} P(x) \log \left( \frac{P(x)}{Q(x)} \right) \quad (15)$$

this shows how far the probability distribution of  $P(x)$  is from the distribution of  $Q(x)$ . KL cannot be used as a general way of inspecting the divergence of two distributions, but it is necessarily specific to one distributions divergence from another.

## V. RESULTS

Using autoencoders the AE achieves the lowest train loss and reconstruction MSE (Mean Square Error), while VAE and CVAE obtain similar values between them and slightly higher compared to AE. Further information can be seen in table II. TABLE II: Train Loss and Test MSE measurements between autoencoder variants

| Model       | Train Loss | Test MSE |
|-------------|------------|----------|
| <b>AE</b>   | 0.044      | 0.047    |
| <b>VAE</b>  | 0.062      | 0.06     |
| <b>CVAE</b> | 0.062      | 0.061    |

Further analysis for train loss and test MSE per epoch can be seen in figures 1a and 1b.

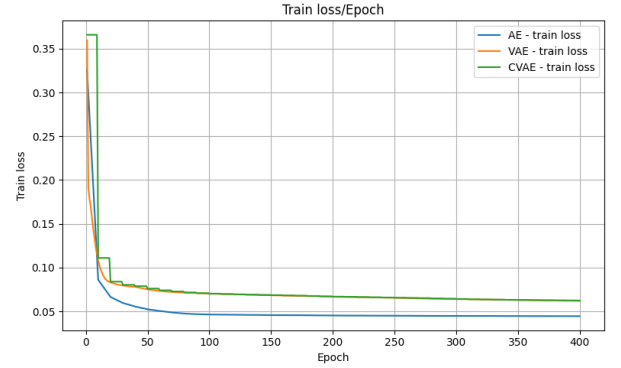
In BCE GAN and WGAN, it is clear that there is convergence between the discriminator and the generator. This indicates, that the generator can trick the discriminator by creating synthetic data that is close to the real data in distribution. Fig. 2a shows great convergence at the start of the model training. However, this changes around 115 epochs where the generator starts having a severe loss. Although it seems that

the data didn't suffer due to the loss, which can be seen in the histogram found in fig 2a. Meanwhile 2b shows that WGAN starts with a heavy adversarial relationship between the generator and the discriminator, until around epoch 40 both them reach to an equilibrium.

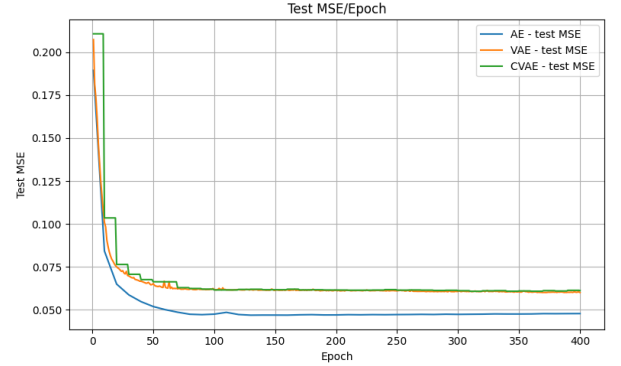
TABLE III: KL Divergence among the different models

| Model          | Min.  | Median | Max    |
|----------------|-------|--------|--------|
| <b>AE</b>      | 0.097 | 0.354  | 16.567 |
| <b>VAE</b>     | 0.438 | 1.715  | 16.931 |
| <b>CVAE</b>    | 0.214 | 1.712  | 19.736 |
| <b>BCE</b>     | 0.054 | 2.428  | 36.517 |
| <b>WGAN-GP</b> | 1.022 | 4.483  | 16.207 |

The histogram in 2b shows, satisfying distribution of the synthetic data. Lastly, by using KL Divergence in both autoencoders and GAN methods compared with the real data, it can be seen that all synthetic distributions are in acceptable margins compared with the real distribution which can be seen in fig. 3a and fig. 3b. The minimum, median and maximum KL-Divergence between the models can be seen in table III.

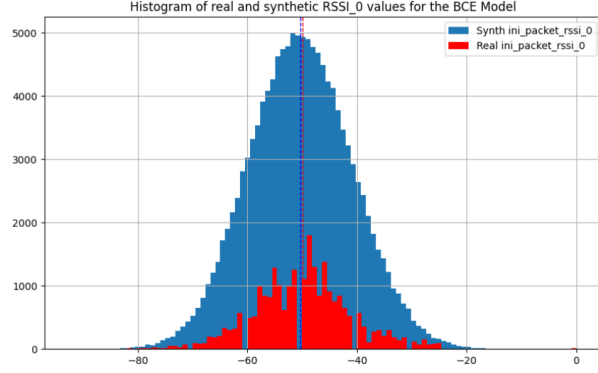
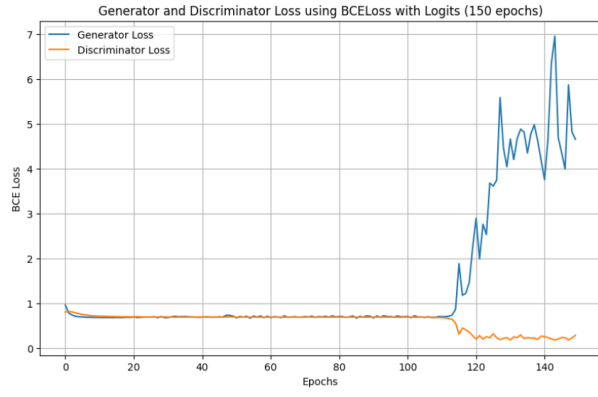


(a) Train loss per epoch for generative autoencoders

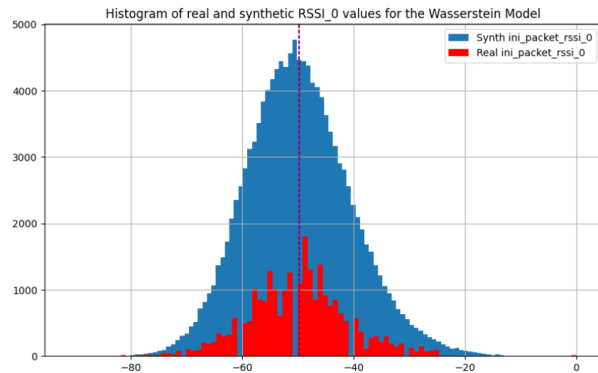
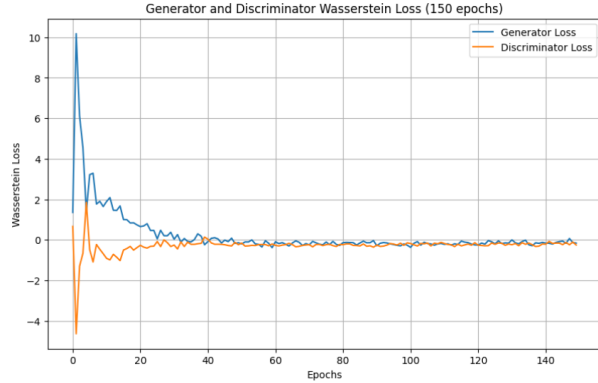


(b) Test loss (MSE) per epoch for generative autoencoder variants

Fig. 1: Training and testing performance of generative autoencoder models

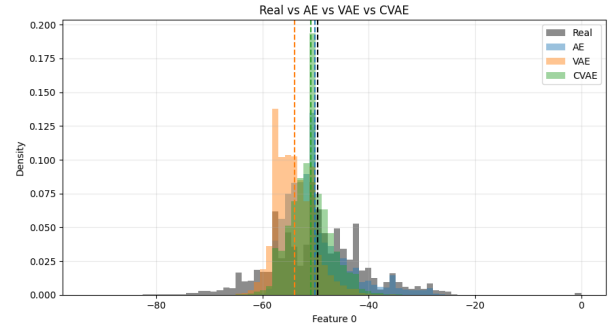


(a) BCE-GAN loss and synthetic data distribution comparison

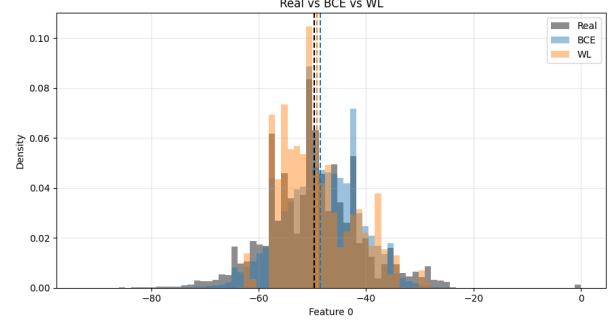


(b) Wasserstein-GAN loss and synthetic data distribution comparison

Fig. 2: GAN Losses and Histograms comparing synthetic and real RSSI<sub>0</sub>, as an inspection metric



(a) Autoencoder-based methods (AE, VAE, CVAE)



(b) GAN-based methods (BCE-GAN, WL-GAN)

Fig. 3: KL divergence between real data and synthetic data generated by different generative models.

## VI. CONCLUSIONS

This paper shows that both autoencoders and GAN methods are capable of creating life-like synthetic data that can be used for data expansion. It is possible that future work could improve on standard deviation and mean of the synthetic data, to bring it even more in line with the real data. But as it stands now, the synthetic data is more than appropriate for the purpose of dataset expansion. Lastly it could encourage future work with traditional or other machine learning methods for distance estimation between the synthetic, the original or a hybrid approach by using both.

## ACKNOWLEDGMENT

The group wants to thank Samsung Denmark Research Center (SDRC) for providing the datasets. Moreover the group appreciates the guidance of the supervisors Jesper H. Sorensen at SDRC and Jan Østergaard from AAU.

## REFERENCES

- [1] Bluetooth® channel sounding. Bluetooth. [Online]. Available: <https://www.bluetooth.com/learn-about-bluetooth/feature-enhancements/channel-sounding/>
- [2] A. H. Mimonah Al Qathrady, "Improving ble distance estimation and classification using tx power and machine learning: A comparative analysis," pp. 79 – 83, 2017. [Online]. Available: <https://dl.acm.org/doi/10.1145/3127540.3127577>
- [3] A. Tsemko, A. Santra, O. Kapshii, and A. Pandey, "Data-driven processing using parametric neural network for improved bluetooth channel sounding distance estimation," pp. 1–5, 2025.
- [4] J. Wieme, R. Nietvelt, D. van Leemput, M. Weyn, P. Crombez, R. Berkvens, E. de Poorter, and J. Hoebeke, "Performance evaluation of bluetooth channel sounding on commercial hardware," *IEEE Access*, vol. 13, pp. 180 862–180 876, 2025.

- [5] M. Afaneh. Introducing channel sounding: Secure fine-ranging using bluetooth le. NovelBits. [Online]. Available: <https://novelbits.io/introduction-to-channel-sounding/>
- [6] G. Tingur. Optimizing indoor localization using rssi and iq data with machine learning. Roger Williams University. [Online]. Available: [https://docs.rwu.edu/computerscience\\_theses/1](https://docs.rwu.edu/computerscience_theses/1)
- [7] What is data augmentation? techniques, examples benefits. CCSLearningAcademy. [Online]. Available: <https://www.ccslearninacademy.com/what-is-data-augmentation/>
- [8] X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang, "Self-supervised learning: Generative or contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 1, pp. 857–876, 2023.
- [9] E. S. Lior Rokach, Oded Maimon, *Machine Learning for Data Science Handbook*. Springer, 2023.
- [10] F. Lachehab, M. Benzaoui, S. A. Tadjer, A. Bensmaine, and H. Hama, "Lstm-autoencoder deep learning model for anomaly detection in electric motor," *Energies*, vol. 17, 2024. [Online]. Available: <https://www.mdpi.com/1996-1073/17/10/2340>
- [11] E. A. B. d. S. S. L. N. Lucas Pinheiro Cinelli, Matheus Araújo Marins, *Variational Methods for Machine Learning with Applications to Deep Networks*. Springer, 2020.
- [12] R. D'Cunha. Conditional variational autoencoders for text to image generation. towardsdata science. [Online]. Available: <https://towardsdatascience.com/conditional-variational-autoencoders-for-text-to-image-generation-1996da9cefcb/>
- [13] S. L. Artidoro Pagnoni, Kevin Liu, "Conditional variational autoencoder for neural machine translation," *arXiv*, 2018. [Online]. Available: <https://arxiv.org/abs/1812.04405>
- [14] J. Langr and V. Bok, *GANs in Action - Deep Learning With Generative Adversarial Networks*, 1st ed. Manning Publications Co., 2019.
- [15] Pytorch - bcewithlogitloss. PyTorch. [Online]. Available: <https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html>
- [16] J. B. PhD. A gentle introduction to generative adversarial network loss functions. Machine Learning Mastery. [Online]. Available: <https://machinelearningmastery.com/generative-adversarial-network-loss-functions/>